

Text Retrieval and Search Engines



Lecture 11 – Passage Retrieval

Search



- Some slides adapted from Stanford

Challenges with Document Retrieval

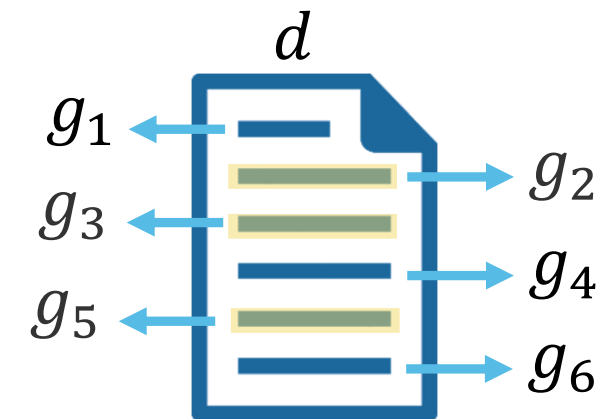
- Documents can be long and topically heterogeneous
- Growing popularity of using mobile search applications



The Focused (Passage) Retrieval Task

Ranking **passages** of documents by their **relevance** to the information need expressed by a query [Geva et al. '10, Arvola et al. '11]

- A **passage** is a (short) sequence of text in a document
- **Focused relevance judgments**
 - Markups of relevant text in relevant documents
 - Example
 - Document d
 - Passage $g_i \in d$



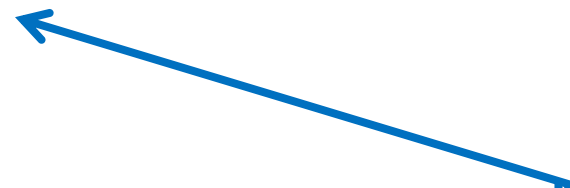
Passage Annotations

Query: “nobel prize”

Relevant document:

Character Offset: 167

Relevance Character Length: 538



“...Science and engineering awards Organizations based in Sweden Awards Nobel Prize Infobox award <http://nobelprize.org> Outstanding contributions in Physics, Chemistry, Literature, Peace, and Physiology or Medicine. The Sveriges Riksbank Prize in Economic Sciences in Memory of Alfred Nobel, commonly identified with the Nobel Prize, is awarded for outstanding contributions in Economics. The Nobel Prize 1901 Swedish Academy Royal Swedish Academy of Sciences Karolinska Institutet Norwegian Nobel Committee Sweden, (Norway) Alfred Nobel. The Nobel Prize (Swedish : Nobelpriset) (Norwegian : Nobelprisen) is a Swedish prize, established in the 1895...”

Applications Of Passage Retrieval

- Passage-based document retrieval
[Callan '94, Wilkinson '94, Kaszkiel&Zobel '97 '01, Liu&Croft '02, Na et al. '08, Bendersky&Kurland '10]
 - Relevant documents can contain much non-relevant information
- Focused retrieval
[Murdock '06, Buffoni et al. '10, Fernández et al. '11, Fernández&Losada '12, Carmel et al. '13]
 - Browsing long documents is a time consuming task
- Applications which use passage retrieval as an intermediate step
 - E.g., question answering, summarization, context for LLM
[Cardie et al. '00, Light et al. '01, Corrada-Emmanuel et al. '03, Tellex et al. '03, Murdock&Croft '04, Aktolga et al. '11, Chen et al. '17, Ram et al. '23]



Different Types Of Passages

- Discourse passages
[Salton et al. '93, Callan '94, Buffoni et al. '10, Losada '10, Fernandez et al. '11 , Carmel et al. '13, Yang et al. '16]
- Semantic passages
[Hearst '93, Hearst&Plaunt. '93, Mittendorf&Schauble '94, Denoyer et al. '01, Jiang&Zhai '04]
- Sliding windows
[Callan '94, Kaszkiel&Zobel '97,'01,Liu&Croft '02, Wang&Si '08, Bendersky&Kurland '10]

Common Passage Ranking Approaches

- Everything that was done for documents can be used for passages!
 - Query likelihood, RM, BM25, etc..

Common Passage Ranking Approaches

- Everything that was done for documents can be used for passages!
 - Query likelihood, RM, BM25, etc..
- However, short relevant passages are prone to vocabulary mismatch with the query
 - Smoothing with the ambient document [Murdock '06, Bendersky&Kurland '08]
 - Smoothing with neighboring passages [Fernández et al. '11, Carmel et al. '13]
 - Query expansion [Losada '10]
 - Using the semantic similarity between a passage and a query; e.g., using entities [Ferragina&Scaiella '12], ESA [Gabrilovich&Markovitch '07], or NNs [Mikolov et al. '13, Nogueira&Cho '20, Karpukhin et al. '20]

Common Passage Ranking Approaches

- Everything that was done for documents can be used for passages!
 - Query likelihood, RM, BM25, etc..
- However, short relevant passages are prone to vocabulary mismatch with the query
 - Smoothing with the ambient document [Murdock '06, Bendersky&Kurland '08]
 - Smoothing with neighboring passages [Fernández et al. '11, Carmel et al. '13]
 - Query expansion [Losada '10]
 - Using the semantic similarity between a passage and a query; e.g., using entities [Ferragina&Scaiella '12], ESA [Gabrilovich&Markovitch '07], or NNs [Mikolov et al. '13, Nogueira&Cho '20, Karpukhin et al. '20]
- Previous work demonstrated the merits of using inter-document similarities to address the vocabulary mismatch problem for documents
 - Cluster-based methods [Jardine&van Rijsbergen '71, Voorhees '85, Kurland&Lee '04, Liu&Croft '04 '08, Raiber&Kurland '13]

Testing The Cluster Hypothesis With Focused Relevance Judgments

[Sheetrit et al. SIGIR '18]

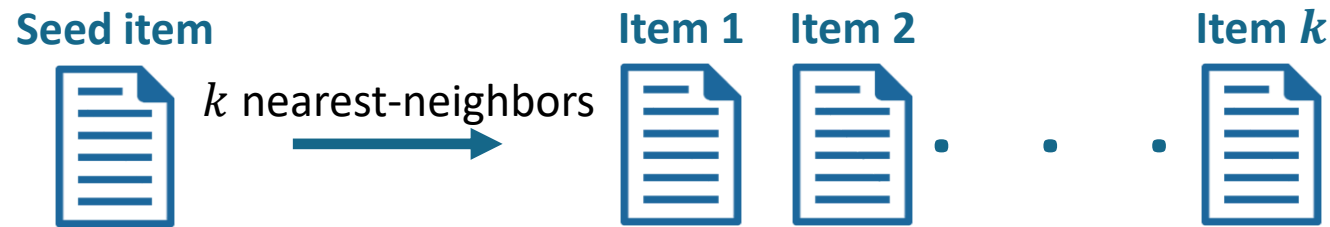
Research questions

1. Does the cluster hypothesis hold for passage retrieval?
2. Does the hypothesis hold to a higher extent for relevant items (documents or passages) containing a high fraction of relevant text?

Testing The Cluster Hypothesis With Focused Relevance Judgments

Based on a paper co-authored with Anna Shtok, Oren Kurland and Igal Shprincis [Sheetrit et al. '18] and on [Sheetrit '18]

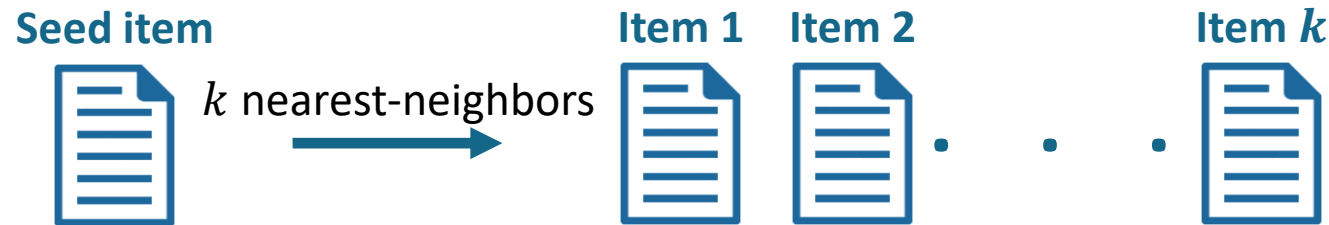
- Based on the nearest-neighbor test [Voorhees '85]
- The tests are performed for a query w.r.t. a list of top retrieved items (documents or passages)
 - β : a relevance degree threshold
 - **Test result**: the average relevance degree of the seed's neighbors



Testing The Cluster Hypothesis With Focused Relevance Judgments

Based on a paper co-authored with Anna Shtok, Oren Kurland and Igal Shprincis [Sheetrit et al. '18] and on [Sheetrit '18]

- Based on the nearest-neighbor test [Voorhees '85]
- The tests are performed for a query w.r.t. a list of top retrieved items (documents or passages)
 - β : a relevance degree threshold
 - **Test result**: the average relevance degree of the seed's neighbors



Binary relevance judgments

- Relevance degrees:
0 (not relevant) or 1 (relevant)
- Seed selection: $\beta = 1$

Focused relevance judgments

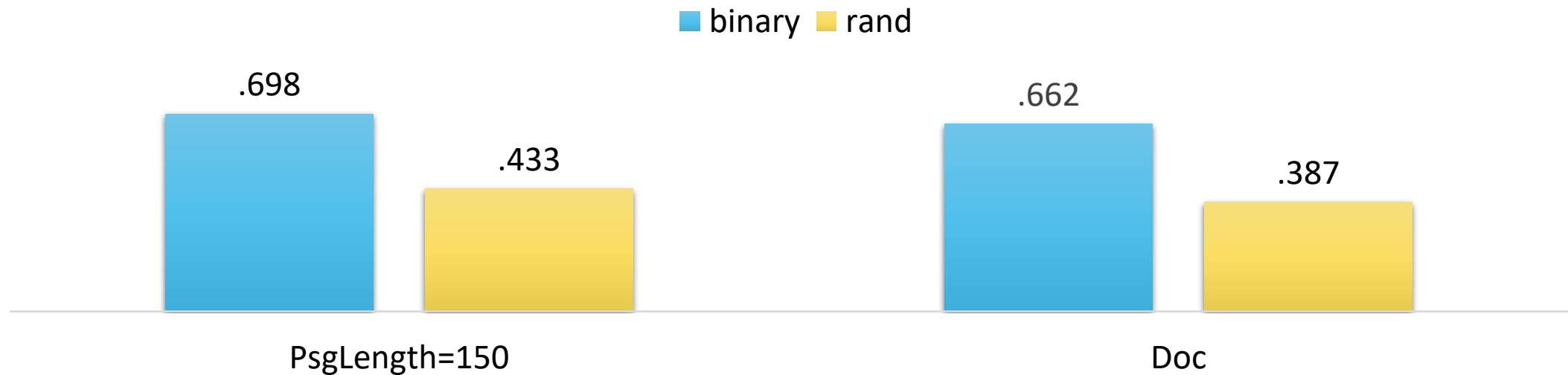
- Relevance degrees:
the item's fraction of relevant text
- Seed selection: $\beta = 0^+$ (rel. degree > 0)

Dataset

Dataset	# of docs	# of queries	Avg rel. fraction	Rel. judgments
INEX	2,666,190	120	.40	Focused

Binary Test Over The INEX Dataset

The average fraction of relevant neighbors of a seed
(4 neighbors are considered); cf. Voorhees '85



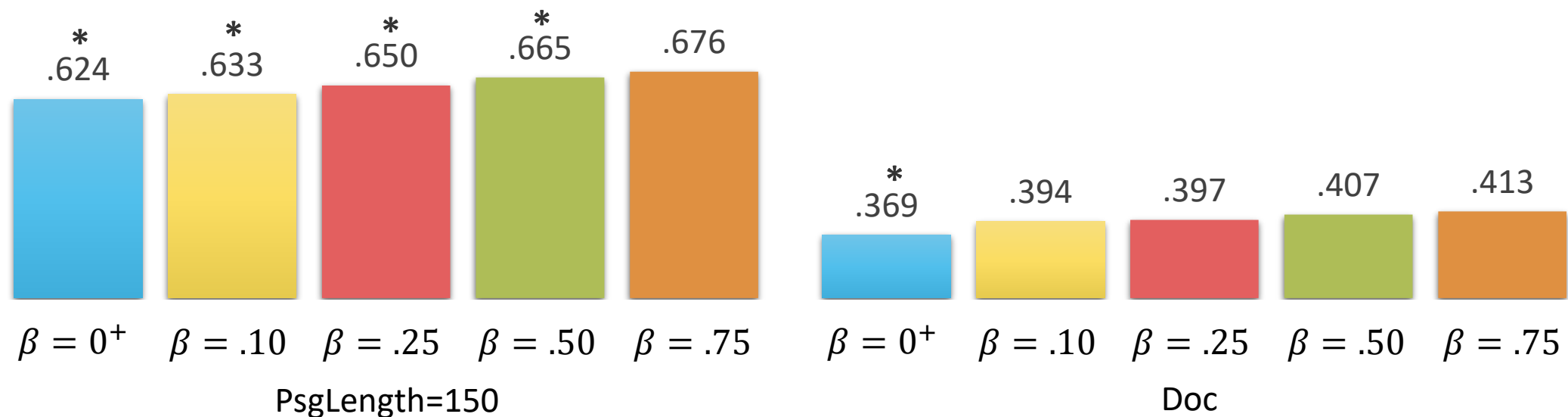
“rand”: random selection of neighbors

Finding:

The hypothesis holds for passage retrieval as determined using the nearest-neighbor test

Focused Test Over The INEX Dataset

The average fraction of relevant text in the seed's 4 nearest-neighbors



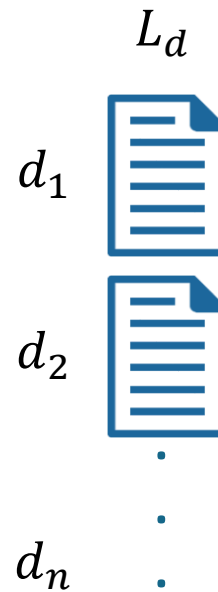
‘*’: statistically significant difference with $\beta = .75$

Finding:

The hypothesis holds to a higher extent for items containing a high fraction of relevant text

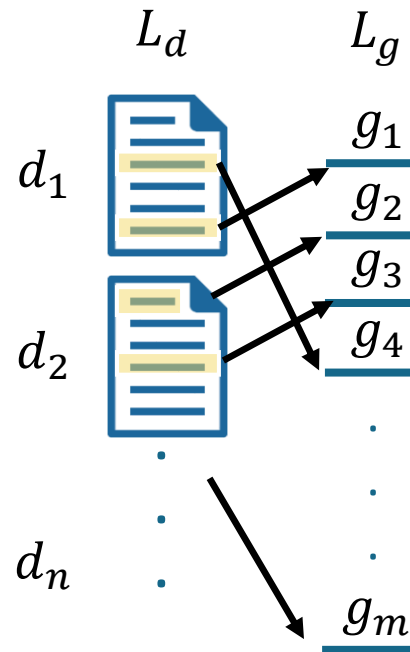
Our ClustPsg Method

- Notation:
 - L_d : List of top ranked documents



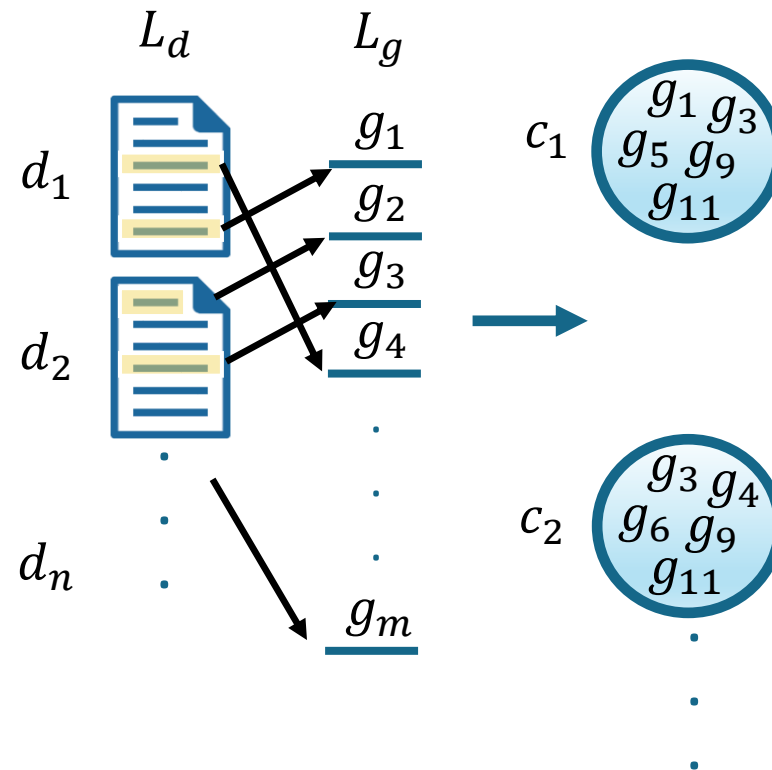
Our ClustPsg Method

- Notation:
 - L_d : List of top ranked documents; L_g : List of top ranked passages;



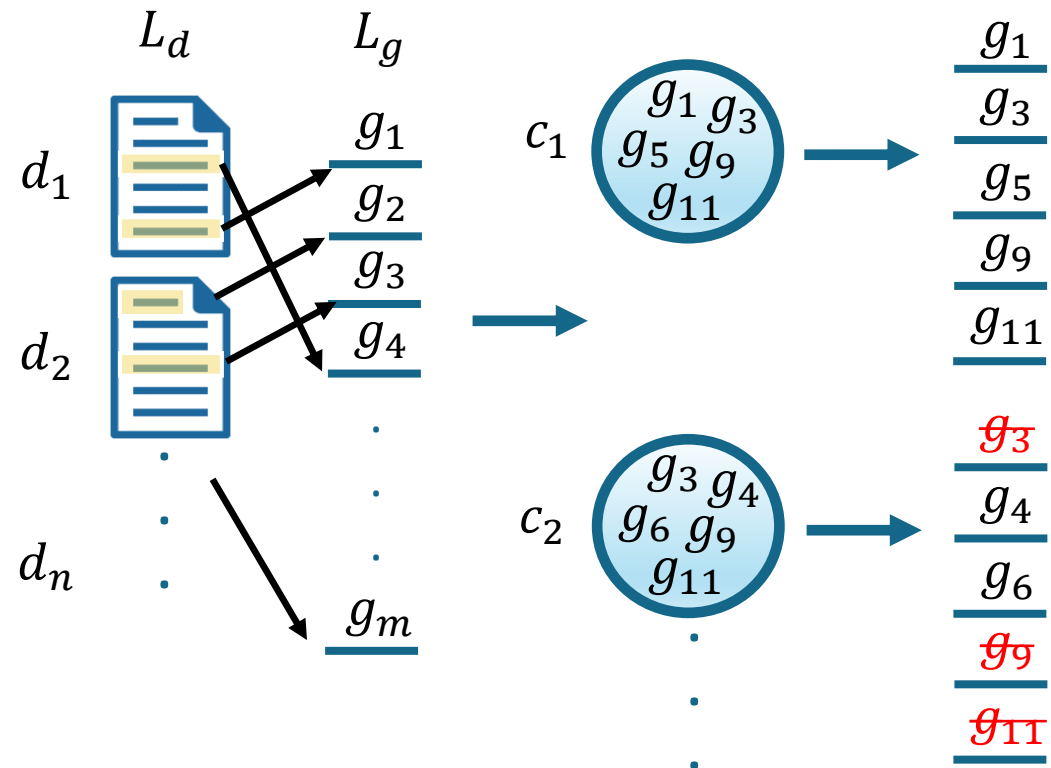
Our ClustPsg Method

- Notation:
 - L_d : List of top ranked documents; L_g : List of top ranked passages;
- Nearest neighbor clustering [Voorhees '85, Sheetrit et al. '18]
- Learning-to-rank approach
 - 2 query dependent features
 - Reciprocal rank position of the cluster's passages in L_g
 - Reciprocal rank position of the passages' documents in L_d
 - 4 cluster priors
 - Entropy of the term distributions
 - Stopwords
 - Inter-passage similarities in the cluster
 - # of unique documents whose passages are in the cluster



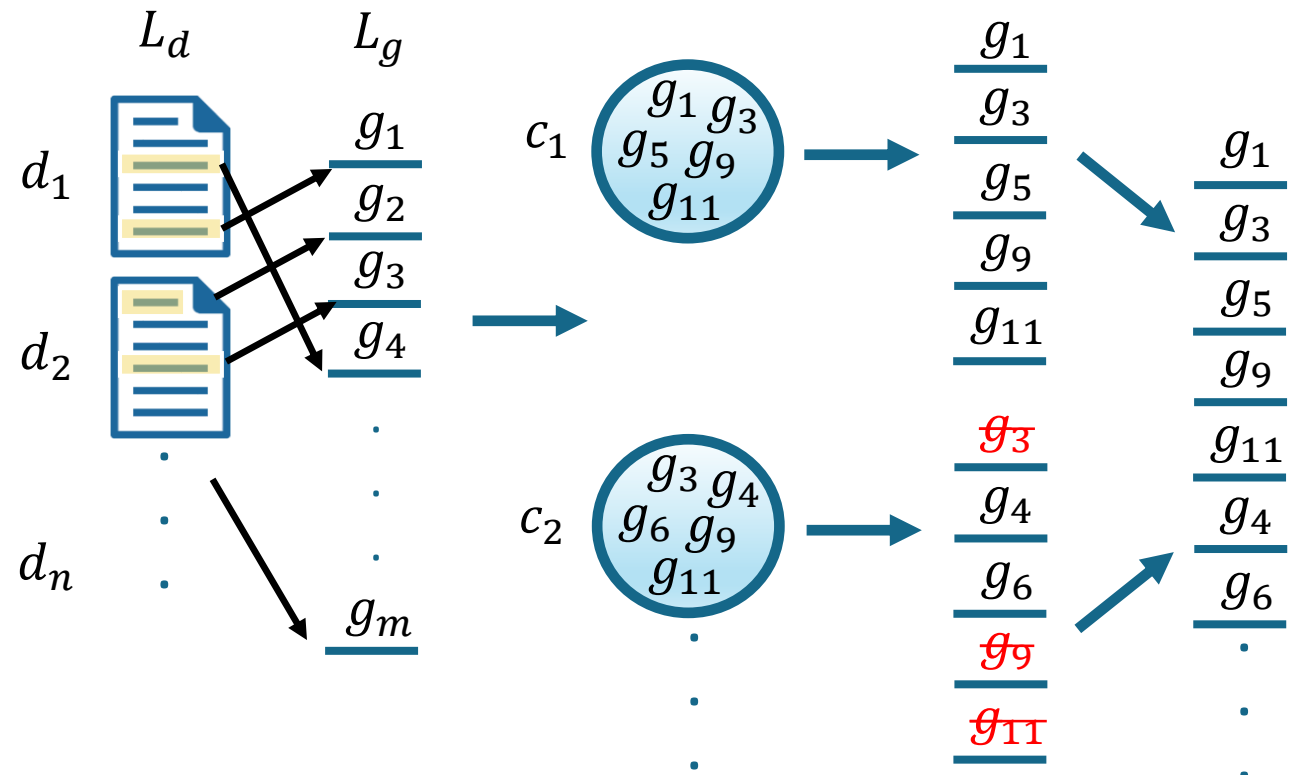
Our ClustPsg Method

- Notation:
 - L_d : List of top ranked documents; L_g : List of top ranked passages;
- Nearest neighbor clustering [Voorhees '85, Sheetrit et al. '18]
- Learning-to-rank approach
 - 2 query dependent features
 - Reciprocal rank position of the cluster's passages in L_g
 - Reciprocal rank position of the passages' documents in L_d
 - 4 cluster priors
 - Entropy of the term distributions
 - Stopwords
 - Inter-passage similarities in the cluster
 - # of unique documents whose passages are in the cluster



Our ClustPsg Method

- Notation:
 - L_d : List of top ranked documents; L_g : List of top ranked passages;
- Nearest neighbor clustering [Voorhees '85, Sheetrit et al. '18]
- Learning-to-rank approach
 - 2 query dependent features
 - Reciprocal rank position of the cluster's passages in L_g
 - Reciprocal rank position of the passages' documents in L_d
 - 4 cluster priors
 - Entropy of the term distributions
 - Stopwords
 - Inter-passage similarities in the cluster
 - # of unique documents whose passages are in the cluster



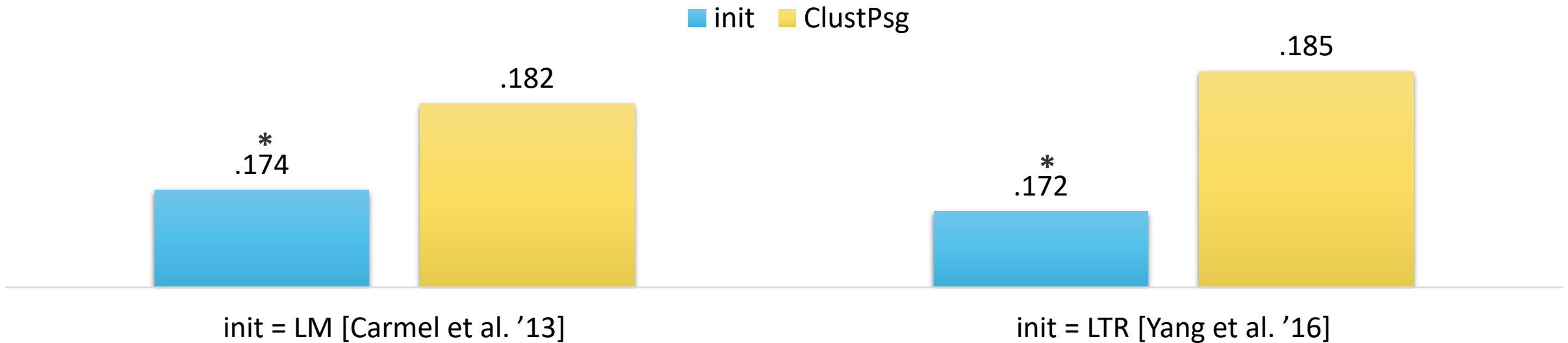
Experiments

- ClustPsg was applied on several initial passage lists L_g created by different retrieval methods, *init*
- Datasets

Dataset	# of docs	# of queries	Avg rel. fraction	Rel. judgments	Passage type
INEX	2,666,190	120	.40	Focused	Windows of 300 terms
AQUAINT	1,033,461	100	.37	Binary	Marked sentences

Main Results over the INEX Dataset

The MAiP of the top 100 ranked passages.
For ClustPsg we use clusters of 10 passages



‘*’: statistically significant difference with ClustPsg

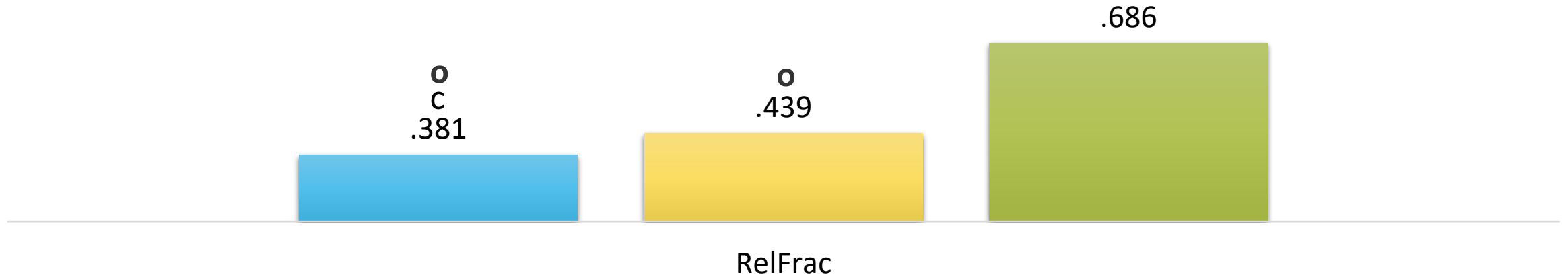
Findings:

1. Query-independent priors are effective for passage retrieval
2. ClustPsg is highly effective in re-ranking different initially retrieved lists

The Optimal Cluster over the INEX Dataset

The average fraction of relevant text (RelFrac)
in the top-10 retrieved passages

■ LM-based ■ ClustPsg ■ Oracle

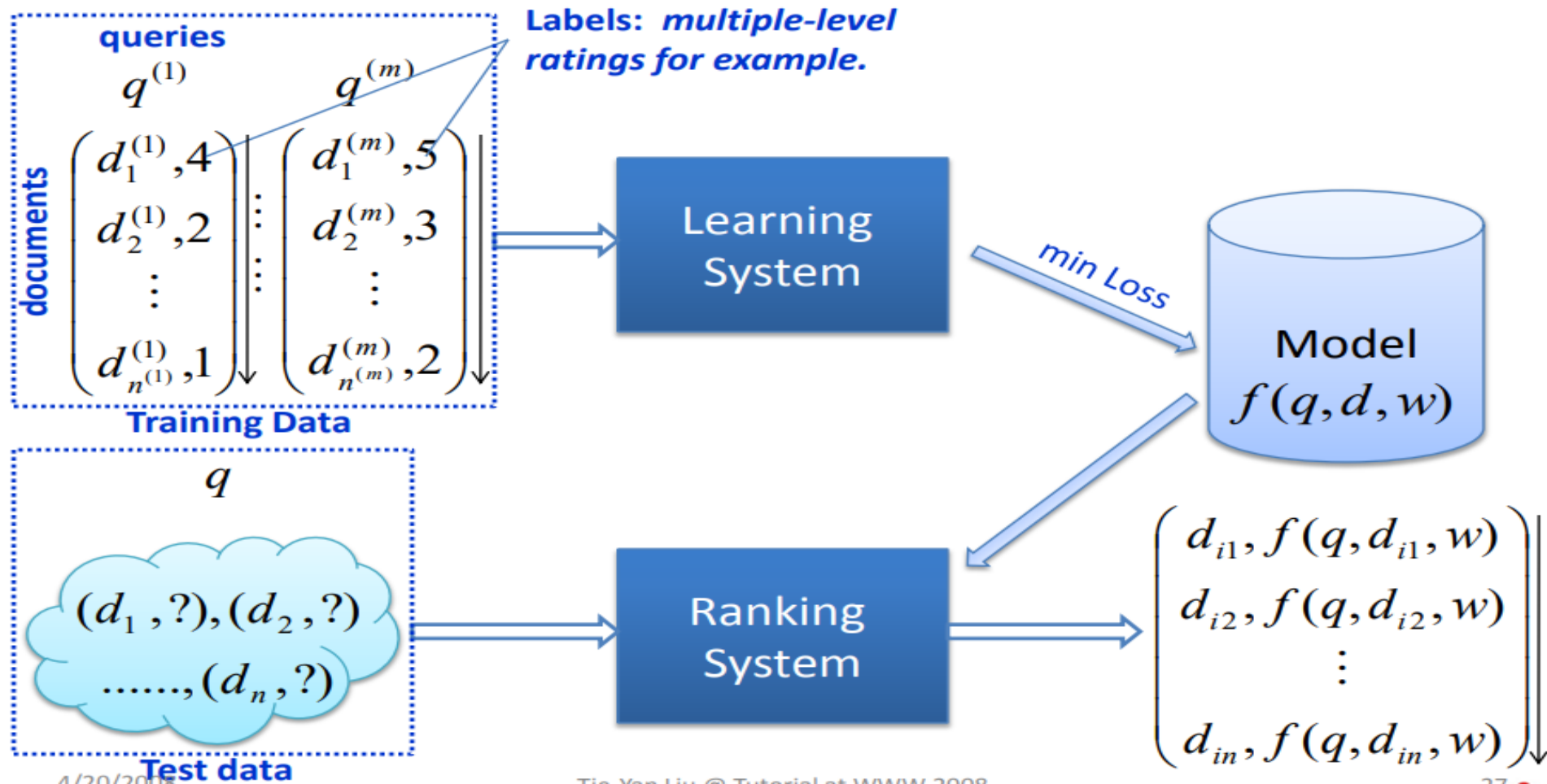


- **Oracle** – Ranking clusters by fraction of relevant text
- 'c' + 'o': statistically significant difference with ClustPsg and Oracle, respectively

Why Learning to Rank?

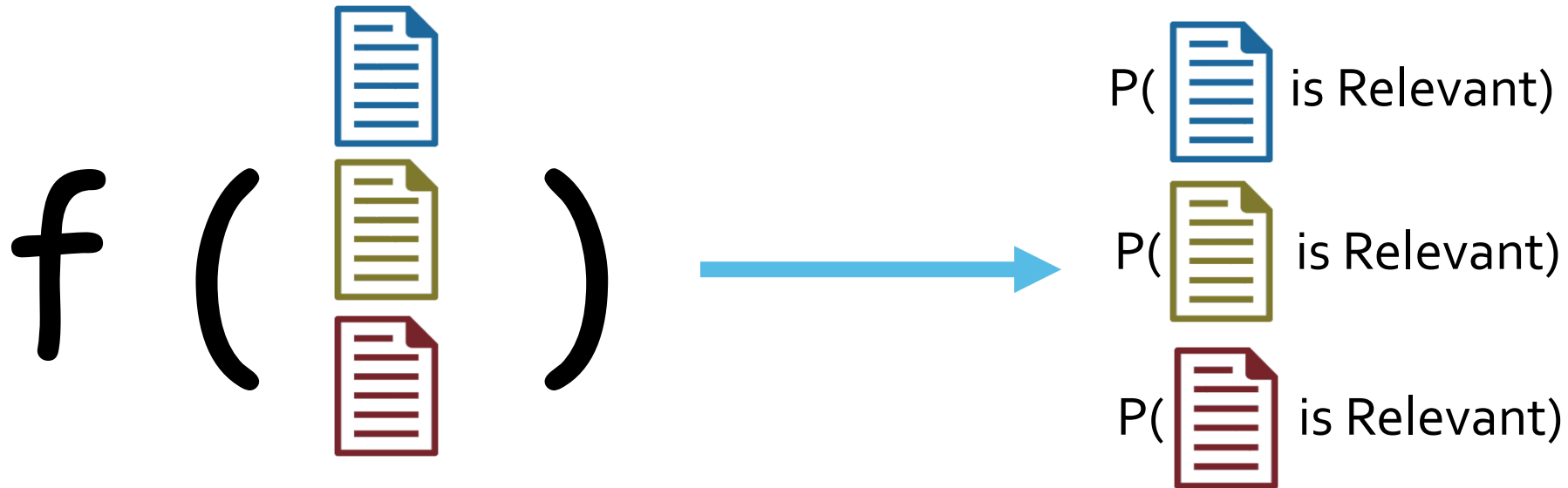
- Machine learning for information retrieval
- From textual to numeric representation
- Corporation of features and relevance signals
 - Handcrafted features based on <query, document> pairs

General Framework



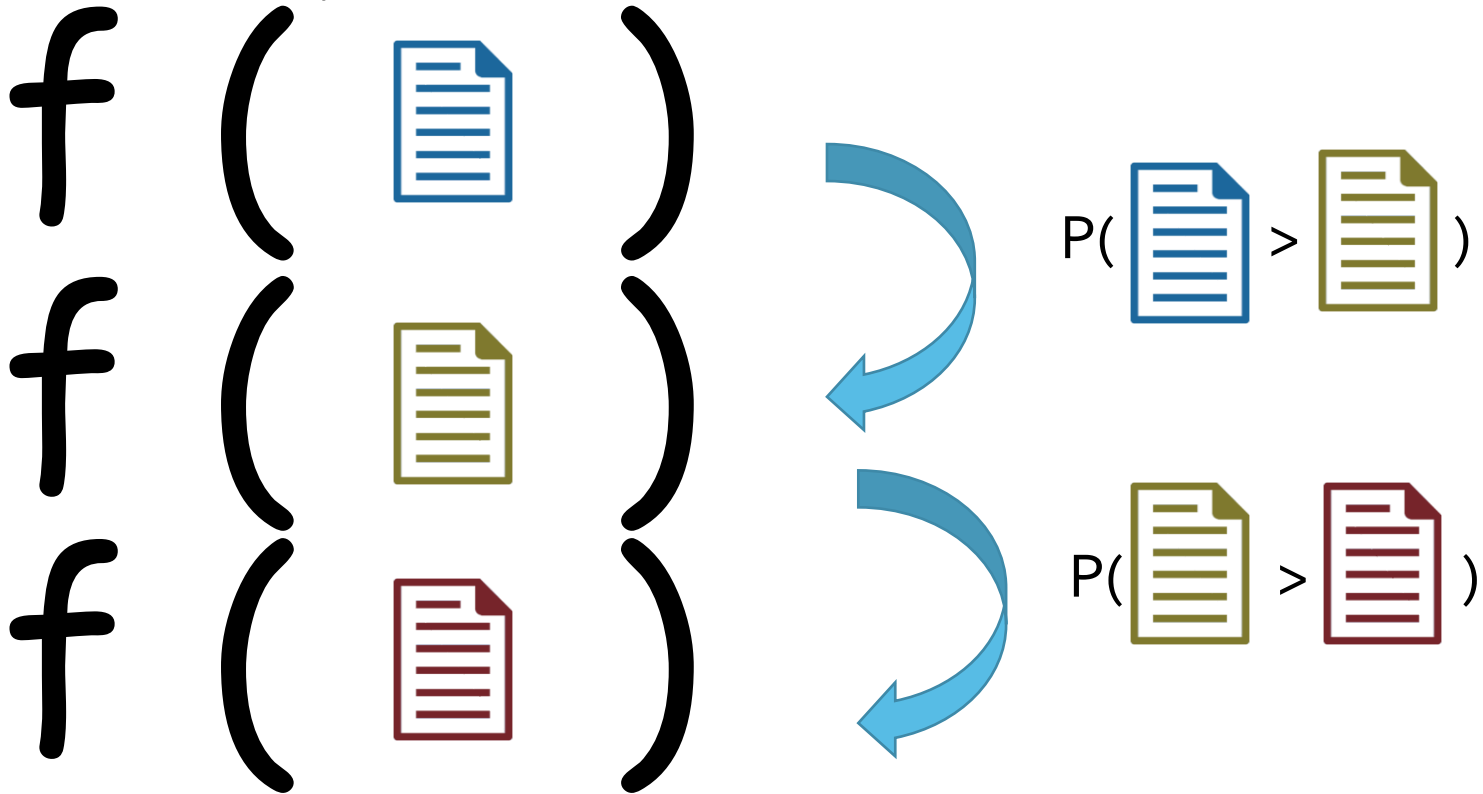
Pointwise Ranking

- Documents are considered independently of each other
 - Regression, classification
- $f(w, V_{(d,q)}) \rightarrow y$
 - w is a vector of model weights, y is the predicted value (relevance judgement)



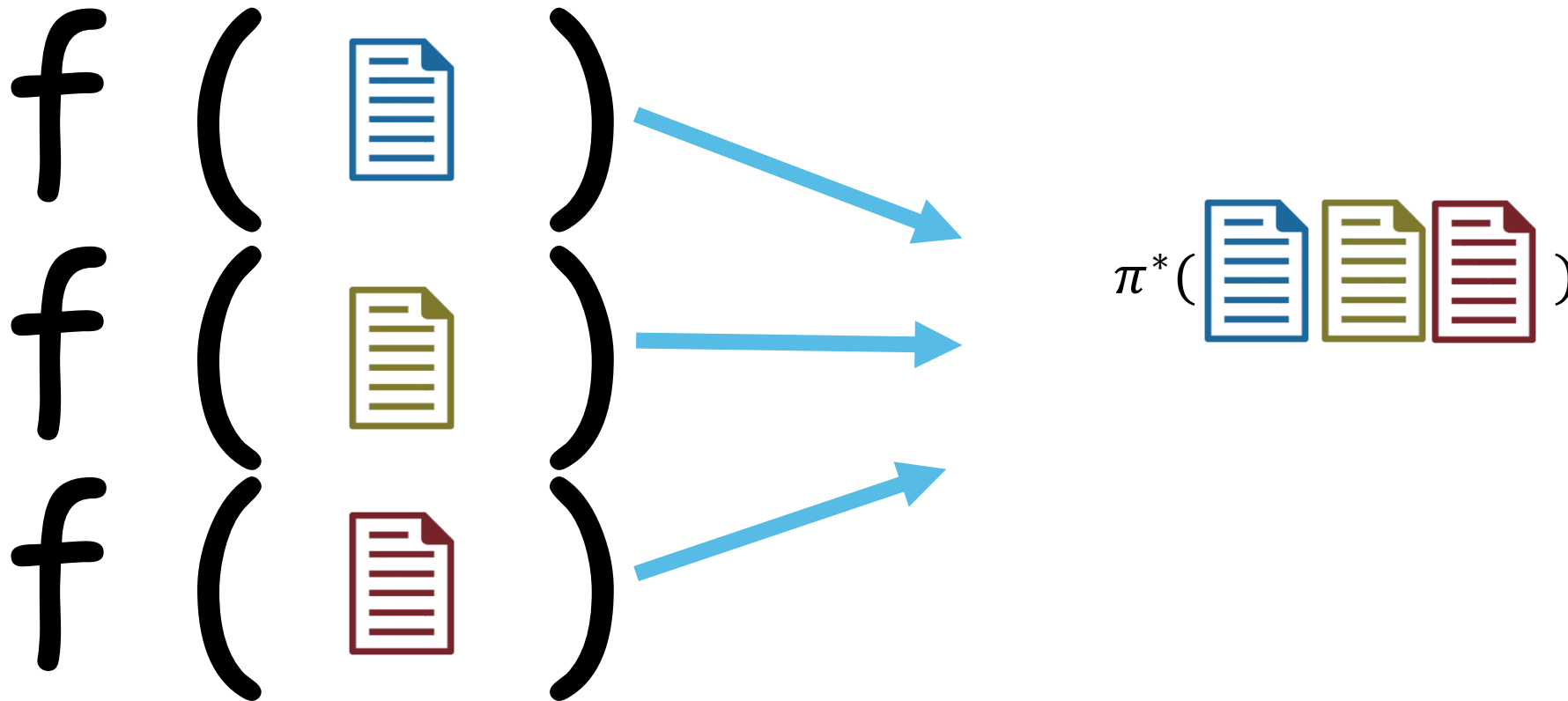
Pairwise Ranking

- Document pairs are considered
 - $f(w, V_{(d_{ij}, q_i)}) \geq f(w, V_{(d_{ik}, q_i)}) + \varepsilon$
 - RankNet, RankSVM



Listwise Ranking

- Consider the ordering of the entire list
 - ListNet



Live Example

```
train.dat x
1 # query 1
2 3 qid:1 1:1 2:1 3:0 4:0.2 5:0
3 2 qid:1 1:0 2:0 3:1 4:0.1 5:1
4 1 qid:1 1:0 2:1 3:0 4:0.4 5:0
5 1 qid:1 1:0 2:0 3:1 4:0.3 5:0
6 # query 2
7 1 qid:2 1:0 2:0 3:1 4:0.2 5:0
8 2 qid:2 1:1 2:0 3:1 4:0.4 5:0
9 1 qid:2 1:0 2:0 3:1 4:0.1 5:0
10 1 qid:2 1:0 2:0 3:1 4:0.2 5:0
11 # query 3
12 2 qid:3 1:0 2:0 3:1 4:0.1 5:1
13 3 qid:3 1:1 2:1 3:0 4:0.3 5:0
14 4 qid:3 1:1 2:0 3:0 4:0.4 5:1
15 1 qid:3 1:0 2:1 3:1 4:0.5 5:0
16
```

```
test.dat x
1 4 qid:4 1:1 2:0 3:0 4:0.2 5:1
2 3 qid:4 1:1 2:1 3:0 4:0.3 5:0
3 2 qid:4 1:0 2:0 3:0 4:0.2 5:1
4 1 qid:4 1:0 2:0 3:1 4:0.2 5:0
5
```

Useful Libraries

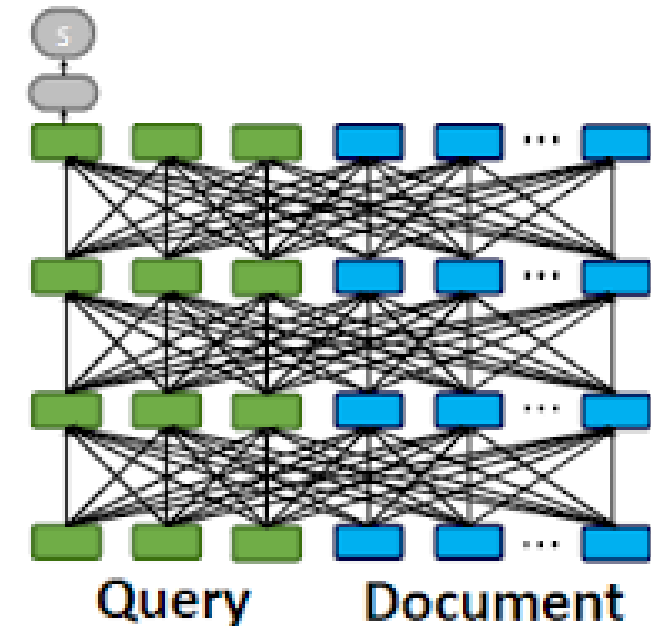
- RankSVM: http://www.cs.cornell.edu/people/tj/svm_light/svm_rank.html
- RankLib: <https://sourceforge.net/p/lemur/wiki/RankLib/>
- TFRanking: <https://github.com/tensorflow/ranking>
- LightGBM: <https://lightgbm.readthedocs.io/en/latest/>

Dense Retrieval

- Represents queries and passages as dense, low-dimensional vectors using neural networks
- Techniques
 - Bi-encoders
 - Cross-encoders
 - LLMs
 - Pointwise / pairwise / listwise
- Hardware
 - GPUs and TPUs made dense methods feasible
- User expectations
 - More natural queries (longer, conversational) demand semantic understanding

All-to-All Interaction with BERT

- Cross encoder + pointwise ranking
- Retrieve the top K documents using a sparse retrieval method (e.g., BM25)
- Feed BERT: “[CLS] Query [SEP] Document [SEP]”
- Run this through all the BERT layers
- Extract the final [CLS] output embedding
 - Classification token, whose output embedding will be used to compute the relevance score
- The [CLS] representation is fed into a fully connected layer
 - Task-specific head is added for text classification



(c) All-to-all Interaction
(e.g., BERT)

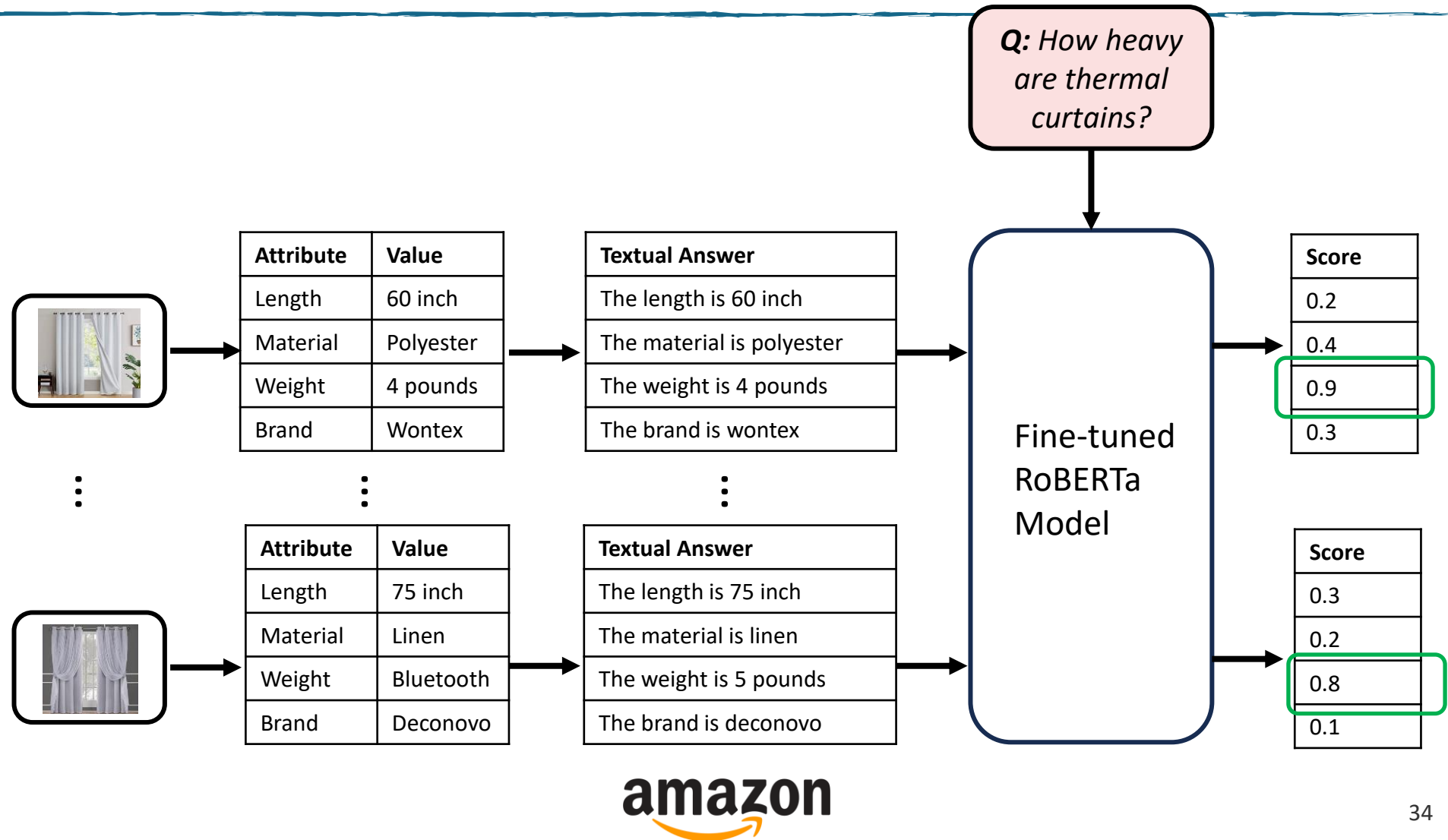
Fine-Tuning Step

[Based on original work by Nogueira&Cho '20]

- Retrieve 1K passages using BM25 for each query
 - Hard negative examples via BM25 instead of random selection
- Cross-entropy loss
 - $L = - \sum_{j \in J_{pos}} \log(s_j) - \sum_{j \in J_{neg}} \log(1 - s_j)$
 - J_{pos} and J_{neg} are the set of retrieved relevant and non-relevant passages, respectively
 - s_j is the relevance score assigned by the model

Case Study - Aggregation-Based Answering For Broad Product Questions

[Sheetrit et al '23]

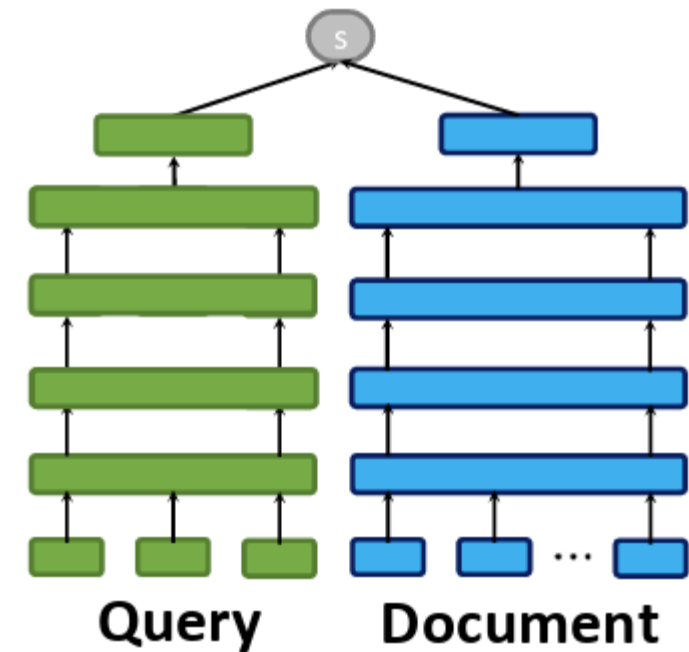


Toward Faster Ranking: Pre-Computation

- BERT rankers are slow because their computations be **redundant**:
 - **Represent the query** (1K times for 1K documents)
 - **Represent the document** (once for every query!)
 - Conduct matching between the query and the document

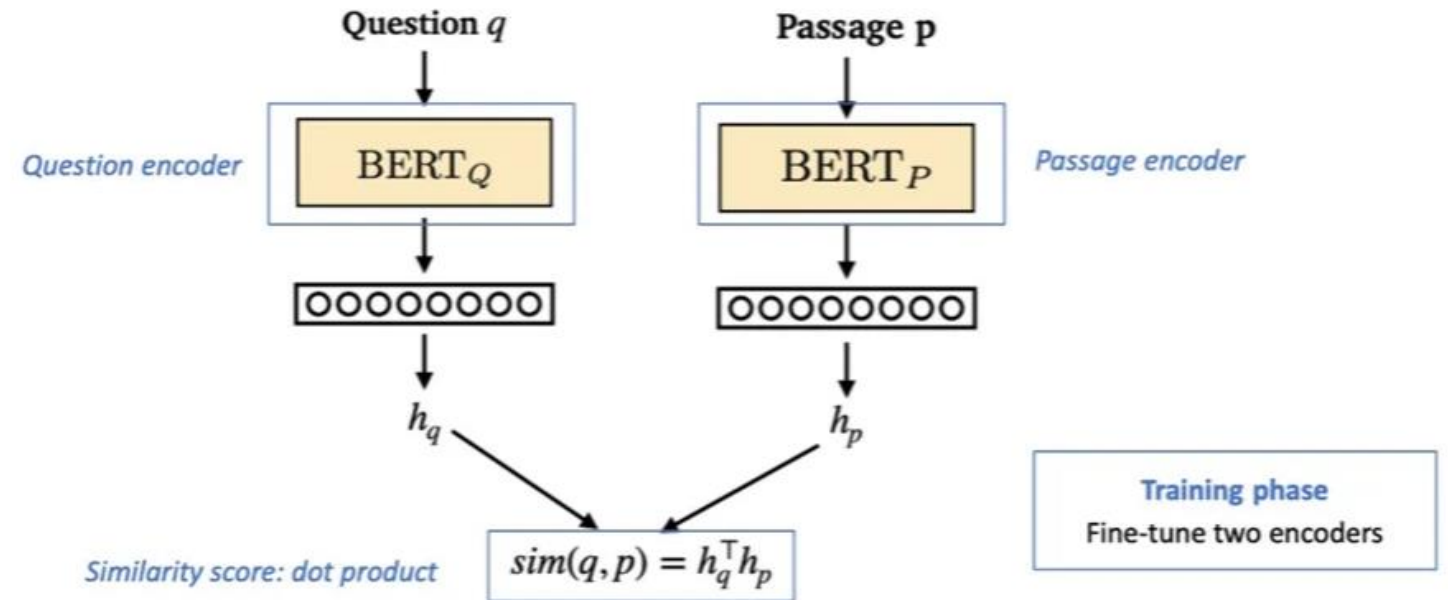
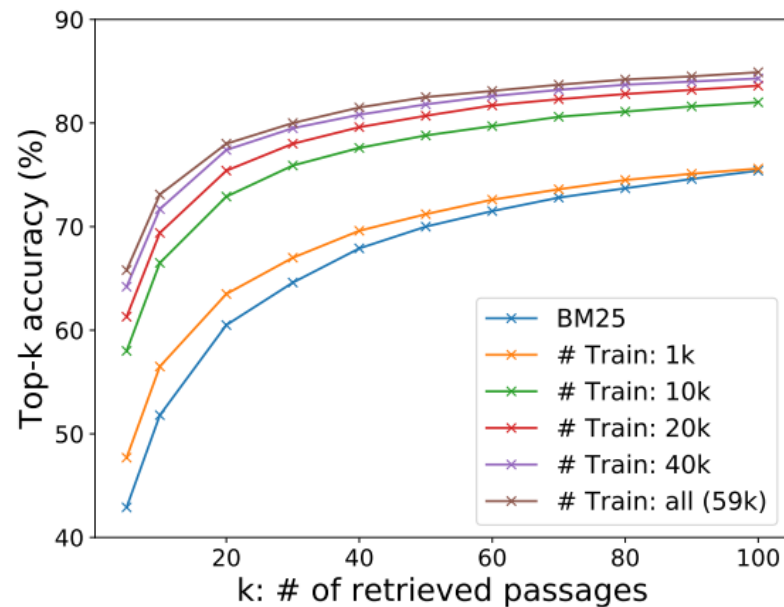
Bi-Encoder

- Tokenize the query and the document
- **Independently** encode the query and the document
 - A single vector representation each
- Estimate relevance using dot product
 - Or a cosine similarity



Bi-Encoder

- E.g., Dense passage retrieval [Karpukhin et al. '20]



Dense Passage Retrieval (DPR)

- Encodes each passage into a 768-dimensional vector
- Encodes each query into a 768-dimensional vector
- Trained with N-way cross-entropy loss, over the similarity scores between the query and:
 - A positive passage
 - A negative passage, samples from BM25 top-100
 - Many in-batch negative passages
 - The positive passages for other queries in the same training batch

$$L(q_i, p_i^+, p_{i,1}^-, \dots, p_{i,n}^-) = -\log \frac{e^{\text{sim}(q_i, p_i^+)}}{e^{\text{sim}(q_i, p_i^+)} + \sum_{j=1}^n e^{\text{sim}(q_i, p_{i,j}^-)}}$$

Representation Similarity: Downsides

- Single vector representation
 - We compress a query and a document to a low dimensional vectors
 - In this compression, fine-grained information is often lost
 - Query tokens never “see” the document tokens during encoding
- No fine-grained interactions
 - Relevance is estimated as **single dot product**
 - We lose **term-level interactions**, which we had in:
 - Query-document interaction models (e.g., BERT)
 - And even term-weighting models (e.g., QL and BM25)

Beyond Re-Ranking: End-to-End Retrieval

- Query-document interaction models forces us to use a re-ranking pipeline, where we just re-scored top-1000 documents retrieved by QL/BM25, ...
 - End-to-end retrieval is essential toward improving recall
- Representation Similarity models alleviate this
 - End-to-end retrieval: quickly searching over all documents directly
 - We can index vector representation for fast vector similarity search

Idea: A “Vector Database” (Vector DB)

- Efficiently index high dimensional data
- Perform vector operations natively
 - Cosine similarity
 - Euclidean distance
 - KNN
- Optimize for Approximate Nearest Neighbor (ANN)
- Handle both sparse and dense vectors efficiently